IronVeil

Framework de Defensa en Profundidad Medible para LLMs Autoalojados

Evaluación Comparativa de Cinco Mecanismos Defensivos frente a los Vectores de Ataque que ya Explotan 175 000 Servidores Expuestos en Internet

Ana Gabriela Fiquitiva Poveda
Anderson Fabian García Nieto
Andrés Felipe Sabogal Wilches
Carlos Mario Piedrahita Arango

Escuela Colombiana de Ingeniería Julio Garavito
Fundamentos de Ciberseguridad
Profesor Diego Alexander López Correa
Octubre de 2026
Bogota D.C., Colombia

Resumen

Miles de instancias del motor de inferencia local Ollama permanecen expuestas en internet sin autenticación, lo que en la práctica habilita a un atacante remoto a extraer *system prompts* completos, desplazarse lateralmente entre modelos relacionados y agotar los recursos del servicio, sin necesidad de credenciales. La literatura reciente documenta que ninguna defensa aislada elimina este riesgo: técnicas de preprocesamiento de entradas alcanzan entre 60 % y 80 % de detección, mientras que arquitecturas más sofisticadas llegan hasta 95 % de protección contra patrones conocidos, pero conservan brechas relevantes frente a atacantes adaptativos. Este proyecto presenta **IronVeil**, un *framework* de defensa en profundidad medible que despliega un laboratorio propio, aislado y controlado sobre Ollama, e implementa cinco mecanismos defensivos independientes y activables por separado (filtrado, delimitación de contenido, clasificación, mínimo privilegio y aprobación humana) para compararlos individualmente y en combinación frente a una batería reproducible de cinco vectores de ataque documentados en la industria y en la academia. A diferencia de las soluciones comerciales que dominan el mercado actual (Prisma AIRS de Palo Alto Networks, FortiAIGate de Fortinet, Lakera Guard de Check Point), orientadas a proteger modelos alojados en servicios cloud gestionados, IronVeil se enfoca deliberadamente en el punto ciego de los despliegues autoalojados y aporta evidencia empírica comparativa –inexistente hoy en el mercado– sobre qué combinación de mecanismos ofrece la mejor relación entre protección, latencia añadida, falsos positivos y costo de implementación. Este documento actualiza la propuesta inicial con lo implementado y medido: sobre cinco corridas en GPU T4 con `llama3.2:1b`, el ASR promedio de los vectores 1 a 4 baja de 31,2 % en el *baseline* a 15,0 % con filtrado solo y con los cinco mecanismos; filtrado, delimitación y mínimo privilegio muestran efecto medible con un costo de latencia inferior a 1 ms, la clasificación añade cerca de 76 ms sin una reducción de fuga distinguible con este modelo, y ninguna configuración llega a 0 %. Como extensiones medidas aparte del núcleo, el trabajo evalúa un clasificador alternativo de pago (Jev, de TypeSafe) frente al clasificador local Llama Prompt Guard 2: en la validación externa con Garak (339 ataques), el clasificador Jev solo dejó pasar 1 de 339 ataques (0,3 %) y, combinado con el resto de mecanismos del proxy, 0 de 339 (intervalo de confianza del 95 % de 0,0 % a 1,1 %), frente a 51,2 % sin defensas; se documenta además la comparación de latencia, cobertura de idiomas y costo (punto de equilibrio de unos 17 millones de peticiones al mes bajo los supuestos declarados). El proxy se distribuye como *framework*: laboratorio completo con un solo comando de Docker Compose, paquete instalable (`pip install ironveil`) y autenticación opcional.

Palabras clave: seguridad en modelos de lenguaje, *prompt injection*, defensa en profundidad, Ollama, OWASP LLM Top 10, *guardrails*, mínimo privilegio, aprobación humana, AI-SPM, SIEM, Jev, Llama Prompt Guard, Garak.

Índice

1 Introducción y Estado del Arte **4**
1.1 El problema: instancias LLM autoalojadas expuestas en internet . . . 4
1.2 Marco de riesgo: OWASP Top 10 para aplicaciones LLM (2025) . . . 4
1.3 Evidencia académica sobre la eficacia de las defensas . . . 4
1.4 Herramientas de la industria: ataque y defensa . . . 5
1.5 Panorama de mercado y diferenciación competitiva . . . 5

2 Problema que se Abordará **7**

3 Pregunta y Objetivos de Investigación **7**
3.1 Pregunta de investigación . . . 7
3.2 Objetivo general . . . 8
3.3 Objetivos específicos . . . 8

4 Alcance **8**
4.1 Dentro del alcance . . . 8
4.2 Fuera del alcance . . . 9

5 Metodología **9**

6 Arquitectura de la Solución **10**
6.1 Diagrama del pipeline de mecanismos activables . . . 10
6.2 Arquitectura por capas: puertos y adaptadores . . . 10
6.3 Componentes opcionales del proxy . . . 10
6.4 Stack tecnológico general . . . 14
6.5 Los cinco mecanismos defensivos . . . 14
6.6 Configuraciones experimentales . . . 16
6.7 Escenarios de prueba: vectores de ataque . . . 16
6.8 Validación de aplicabilidad con modelos frontera . . . 17

7 Resultados Esperados y Métricas de Evaluación **17**
7.1 Métricas transversales . . . 17
7.2 Resultados esperados por configuración . . . 18
7.3 Formato de la tabla final de resultados . . . 19
7.4 Resultados obtenidos . . . 20
7.5 Extensiones medidas fuera del núcleo . . . 21
7.6 Clasificador alternativo: Jev frente a Llama Prompt Guard 2 . . . 21

8 Distribución, Reproducibilidad y Calidad **23**

9 Conclusiones y Trabajo Futuro **23**
9.1 Conclusiones . . . 23
9.2 Limitaciones . . . 24
9.3 Trabajo futuro . . . 24

10 Cronograma Inicial de Trabajo **25**

11 Consideraciones Éticas y Legales **27**

Referencias **28**

1 Introducción y Estado del Arte

1.1 El problema: instancias LLM autoalojadas expuestas en internet

Ollama fue diseñado como una herramienta de uso local y, por esa razón, no incorpora autenticación por defecto. Cuando un equipo de desarrollo configura la variable de entorno `OLLAMA_HOST=0.0.0.0` para permitir el acceso desde otra máquina (algo común en despliegues en la nube o en contenedores), el servicio queda expuesto a cualquier persona en internet sin ninguna barrera de entrada.

La magnitud del problema ha crecido de forma sostenida y bien documentada. Una investigación conjunta de SentinelLabs (SentinelOne) y Censys, que analizó 7,23 millones de observaciones a lo largo de 293 días, identificó 175 108 servidores Ollama únicos (aproximadamente 175 000) expuestos públicamente en 130 países y 4 032 sistemas autónomos, con una porción persistente de alrededor de 23 000 hosts responsables de la mayor parte de la actividad observada (SentinelOne, 2026). Un hallazgo especialmente relevante para justificar este proyecto es que cerca de la mitad de esos servidores (más del 48 %) anuncian la funcionalidad de *tool calling* en su API, es decir, no son simples fugas pasivas de información sino puntos de entrada activos capaces de ejecutar código e interactuar con sistemas externos (SentinelOne, 2026).

En mayo de 2026, investigadores de Cyera divulgaron una vulnerabilidad crítica en Ollama denominada *Bleeding Llama* (CVE-2026-7482, CVSS 9.1): una lectura fuera de límites en el cargador de archivos GGUF que permite a un atacante no autenticado filtrar memoria del *heap* del proceso (claves de API, variables de entorno, *system prompts* y conversaciones de otros usuarios) mediante tres llamadas a la API. Cyera estimó que la vulnerabilidad afectaba a cerca de 300 000 servidores expuestos a nivel global (Cyera Research, 2026); las versiones anteriores a la 0.17.1 son las vulnerables (The Hacker News, 2026). Este hallazgo confirma, con un caso concreto y reciente, el patrón de riesgo que motiva el proyecto: exposición sin autenticación, fuga de *system prompt* y credenciales, y superficie de ataque que crece junto con la adopción de la herramienta.

1.2 Marco de riesgo: OWASP Top 10 para aplicaciones LLM (2025)

La referencia estándar de la industria para clasificar riesgos de seguridad en sistemas basados en LLM es el *OWASP Top 10 for LLM Applications*, publicado por el OWASP GenAI Security Project y actualizado en su edición 2025 (OWASP GenAI Security Project, 2025). *Prompt Injection* (LLM01) mantiene el primer lugar por segunda edición consecutiva: dado que los LLM procesan instrucciones y datos por el mismo canal sin una separación estructural clara, un atacante puede construir una entrada que el modelo interprete como una instrucción nueva en lugar de como contenido a procesar. La edición 2025 también reordenó *Sensitive Information Disclosure* (LLM02) al segundo lugar, incorporó *System Prompt Leakage* (LLM07) como categoría nueva (tras incidentes reales donde se extrajeron las reglas internas y la lógica de filtrado de sistemas en producción) y amplió *Excessive Agency* (LLM06) para cubrir los riesgos de agentes con herramientas y permisos excesivos. Este proyecto se alinea directamente con LLM01, LLM02, LLM06, LLM07 y LLM10 (*Unbounded Consumption*); dentro del alcance quedan explícitamente fuera LLM03 (*Supply Chain*), LLM04 (*Data and Model Poisoning*), LLM08 (*Vector and Embedding Weaknesses*, propio de sistemas RAG) y LLM09 (*Misinformation*), por las razones que se detallan en la sección de Alcance.

1.3 Evidencia académica sobre la eficacia de las defensas

La pregunta que sostiene este proyecto (qué combinación de mecanismos ofrece mejor protección con menor impacto en utilidad y costo) está directamente respaldada por la literatura académica reciente. Una revisión sistemática que sintetiza 128 estudios revisados por pares entre 2022 y 2025, siguiendo las directrices de Kitchenham, encontró que las técnicas de ataque evolucionaron rápidamente de inyecciones directas simples a ataques multimodales sofisticados, logrando tasas de éxito superiores al 90 % contra sistemas sin ninguna

protección. La misma revisión reporta que el preprocesamiento de entradas (una defensa relativamente simple, análoga al mecanismo de *filtrado* de este proyecto) logra entre 60 % y 80 % de tasa de detección, mientras que las defensas arquitectónicas más avanzadas alcanzan hasta 95 % de protección contra patrones de ataque conocidos, pero persisten brechas importantes contra ataques nuevos o adaptativos (Geng, Xu, Qu & Wong, 2026). Esta misma fuente identifica 37 enfoques de defensa distintos agrupados en tres niveles (preprocesamiento de entrada, arquitectura de sistema y nivel de modelo) que corresponden, de forma natural, a los cinco mecanismos que este proyecto aísla y mide por separado.

En la práctica de la industria, Microsoft documenta una estrategia de defensa en profundidad contra la inyección indirecta de *prompts* que combina tres capas: prevención mediante la técnica de *Spotlighting* (marcado estructural del texto no confiable mediante delimitación, marcado de datos o codificación, para que el modelo distinga instrucciones de datos), detección mediante un clasificador (*Prompt Shields*) y salvaguardas deterministas que limitan el impacto incluso cuando la inyección tiene éxito (Microsoft Security Response Center, 2025). La técnica de *Spotlighting* fue propuesta originalmente por Hines et al. (2024) y corresponde directamente al mecanismo de *delimitación de contenido* de este proyecto. En la misma línea, trabajos académicos como StruQ (Chen, Piet, Sitawarin & Wagner, 2025) y SecAlign (Chen et al., 2025) proponen aplicar el principio de *separación de control y datos*, propio de la seguridad de software tradicional, directamente en la arquitectura del LLM.

Sobre el mecanismo de mínimo privilegio, Unit 42, el equipo de inteligencia de amenazas de Palo Alto Networks, recomienda explícitamente limitar a los agentes de IA al mínimo de herramientas y permisos necesarios, y exigir una capa adicional de aprobación humana para las acciones de alto impacto, señalando que "no existe una forma determinista de proteger a los agentes de realizar acciones maliciosas" salvo reducir sus permisos al mínimo indispensable (Unit 42, Palo Alto Networks, 2026a). El mismo equipo advierte que los *guardrails* de contenido son una defensa "blanda" (detectan y bloquean la mayoría de los intentos, pero es actualmente imposible bloquear de forma determinista el cien por ciento de los ataques), mientras que reducir permisos es una defensa "dura" y determinista (Unit 42, Palo Alto Networks, 2026b). Esta distinción entre defensas probabilísticas (filtrado, clasificación) y defensas estructurales (delimitación, mínimo privilegio) es central en el diseño de este proyecto y coincide con la conclusión de la revisión de Geng et al. (2026): ninguna defensa aislada es suficiente, por lo que la pregunta relevante no es cuál mecanismo es mejor, sino qué combinación ofrece la mejor relación entre protección, latencia y usabilidad.

1.4 Herramientas de la industria: ataque y defensa

Del lado ofensivo, las principales herramientas open-source usadas para evaluar sistemáticamente estas vulnerabilidades son Garak (NVIDIA), orientado al escaneo amplio de un modelo con más de 120 pruebas (*probes*); PyRIT (Microsoft), especializado en ataques multiturno con escalamiento conversacional (*crescendo*); y Promptfoo, que permite automatizar pruebas dentro de un pipeline de integración continua. Del lado defensivo, NeMo Guardrails (NVIDIA) es un *framework* de "rieles" de conversación programables que controla el flujo completo del diálogo, y frecuentemente utiliza Llama Guard (Meta), un modelo clasificador de seguridad de contenido, como el componente que ejecuta la validación semántica por debajo. Ambas herramientas no son competidoras sino piezas complementarias, lo cual respalda directamente la premisa de IronVeil: ningún componente cubre por sí solo todas las capas del problema, y la mayoría de sistemas en producción combinan dos o más.

1.5 Panorama de mercado y diferenciación competitiva

Antes de presentar la propuesta, es necesario responder una pregunta que cualquier evaluador con criterio de mercado formulará de inmediato: ¿ya existen productos comerciales para este problema, y en qué se diferencia IronVeil de ellos?

El mercado se está consolidando por adquisiciones, no por consenso técnico. En los últimos dieciocho meses, los grandes proveedores de ciberseguridad compraron a las startups especializadas en seguridad de IA en lugar de construir esta capacidad desde cero: Palo Alto Networks adquirió Protect AI (julio de 2025) y lanzó *Prisma AIRS*, que cubre escaneo de modelos, seguridad de la cadena de suministro, *red teaming* automatizado y protección en tiempo de ejecución (Maxim AI, 2026); Check Point adquirió Lakera, cuyo producto Lakera Guard clasifica *prompts* y respuestas en tiempo real con más de 98 % de tasa de detección y menos de 50 ms de latencia (Wiz, 2026); Cisco adquirió Robust Intelligence; y SentinelOne adquirió Prompt Security. Fortinet, por su parte, no adquirió sino que desarrolló internamente *FortiAIGate*, un *gateway* de seguridad en tiempo de ejecución que se sitúa entre la aplicación y el modelo, aplica *guardrails* sobre la entrada y la salida, y mapea su cobertura explícitamente contra el OWASP Top 10 para LLM (Fortinet, 2026).

Un dato relevante para contextualizar la madurez de esta categoría: a la fecha de este documento **no existe un Cuadrante Mágico de Gartner dedicado específicamente a "seguridad de LLM"** como categoría independiente; un análisis de mercado de abril de 2026 confirma explícitamente que la categoría adyacente de "AI SOC" tampoco cuenta todavía con Cuadrante Mágico ni con Forrester Wave propios (Security Boulevard, 2026). Esto no significa que el mercado esté vacío –como muestra la lista de adquisiciones anterior, está lejos de estarlo– sino que es una categoría emergente sin un estándar de evaluación consolidado, exactamente el tipo de espacio donde un aporte empírico y comparativo como el de IronVeil tiene valor diferencial.

Herramienta	**Qué SÍ resuelve**	**Qué NO cubre (brecha frente a IronVeil)**
Lakera Guard (Check Point)	Clasifica *prompts*/respuestas en tiempo real; funciona con *backends* autoalojados (Wiz, 2026)	Caja negra comercial de pago; no combina 5 mecanismos medibles por separado ni publica evidencia comparativa de qué combinación protege mejor
Prisma AIRS (Palo Alto, ex-Protect AI)	Cubre todo el ciclo de vida: escaneo de modelos, *supply chain*, *red teaming* automático, *runtime* (Maxim AI, 2026)	Orientado a grandes empresas ya licenciadas en el ecosistema Palo Alto; no aísla el efecto individual de cada mecanismo
FortiAIGate (Fortinet)	*Gateway* empresarial con *guardrails*, límite de tasa y mitigación de DDoS, mapeado al OWASP Top 10 LLM (Fortinet, 2026)	Requiere aceleración por hardware (SmartNIC/GPU) y licenciamiento FortiCare; inviable para un laboratorio autoalojado pequeño; no publica comparativas por capa
HiddenLayer	Protege el modelo mismo: detección de robo, *tampering* y manipulación de pesos (Check Point Software, 2026)	No ataca el vector conversacional (inyección de *prompt* vía texto), que es el núcleo de IronVeil
Guías de *hardening* manual (nginx + auth + firewall)	Es la práctica más documentada hoy para exponer Ollama de forma menos insegura (AI Thinker Lab, 2026)	Cubren solo el control de acceso perimetral; ningún tutorial mide ni compara el efecto de capas adicionales
llm-shield (proyecto independiente en GitHub)	*Gateway* + *firewall* + monitoreo específico para IA autoalojada, en fase temprana de desarrollo (JTHil, 2026)	Producto individual de un desarrollador; no implementa mínimo privilegio entre modelos ni aprobación humana, ni evaluación comparativa empírica

> **¿Ya existen "plugins" de solo integrar y listo?:** Sí, para un mecanismo aislado: la clasificación de contenido. Lakera Guard, por ejemplo, es literalmente una API que se integra en pocas líneas de código (Wiz, 2026). Pero ningún producto comercial ni de código abierto combina los cinco mecanismos con mínimo privilegio entre modelos autoalojados relacionados y aprobación humana como capas independientes y medibles por separado, y sobre todo, ninguno publica evidencia empírica comparativa de qué combinación ofrece la mejor relación protección/utilidad/costo. Los proveedores venden su propia solución como la mejor; IronVeil mide, con datos propios, si eso es cierto y bajo qué condiciones.

Finalmente, es importante precisar el terreno específico donde compite IronVeil. La inmensa mayoría de las herramientas comerciales listadas arriba –incluyendo Prisma AIRS, FortiAIGate y los *guardrails* nativos de AWS Bedrock y Azure AI Content Safety– están diseñadas para proteger **modelos alojados en servicios cloud gestionados por el propio proveedor**. El problema que motiva este proyecto (Sección 1.1) –175 000 instancias Ollama expuestas, con 48 % permitiendo ejecución de código (SentinelOne, 2026)– es precisamente el punto ciego que esas herramientas de nube gestionada no cubren, porque el modelo no vive en su infraestructura. IronVeil no compite por el mismo mercado que Prisma AIRS o FortiAIGate: ataca el nicho específico, real y creciente, de los despliegues autoalojados sin gestión centralizada.

2 Problema que se Abordará

Miles de instancias de LLM autoalojados, con Ollama como caso representativo y ampliamente documentado, se exponen accidentalmente en internet sin autenticación, principalmente por errores de configuración durante el despliegue. Esto permite a un atacante extraer *system prompts* completos, realizar movimiento lateral entre modelos relacionados dentro de la misma organización, y agotar los recursos del servicio, todo ello sin necesidad de credenciales válidas.

El problema no es únicamente la ausencia de protección: es que, en la práctica, las organizaciones suelen implementar un único mecanismo de defensa de forma aislada (por ejemplo, solo un filtro de palabras, o solo autenticación de red) sin evidencia clara de qué combinación de los mecanismos disponibles reduce realmente el riesgo sin volver el sistema inutilizable o costoso de mantener. La literatura académica reciente confirma esta brecha: existen decenas de mecanismos de defensa propuestos de forma individual, pero pocos estudios comparan sistemáticamente combinaciones de mecanismos heterogéneos (basados en reglas, en clasificadores, en control de acceso y en supervisión humana) sobre un mismo sistema y con la misma batería de ataques.

Existe, además, una brecha operativa que rara vez se documenta junto con la brecha técnica anterior: incluso cuando un mecanismo de defensa bloquea correctamente un ataque, ese bloqueo suele quedar silencioso. Sin un canal de notificación, el equipo de seguridad no se entera del incidente en el momento en que ocurre; y sin una integración con un sistema de gestión de eventos e información de seguridad (SIEM), ese evento tampoco puede correlacionarse con el resto de la actividad de seguridad de la organización. Una defensa técnicamente efectiva pero operativamente invisible no cierra el ciclo completo de detección y respuesta. Este proyecto aborda también esa brecha mediante una extensión ya implementada: exportación del mismo *log* que generan los cinco mecanismos en formatos JSON, CEF, LEEF y ECS hacia un SIEM (con reglas de Wazuh incluidas) y avisos por *webhook*, Slack o correo cuando la aprobación humana encola una petición (Sección 6).

3 Pregunta y Objetivos de Investigación

3.1 Pregunta de investigación

¿Qué combinación de los cinco mecanismos defensivos (filtrado, delimitación de contenido, clasificación,

mínimo privilegio y aprobación humana) reduce de forma más efectiva la tasa de éxito de los vectores de ataque documentados contra LLM autoalojados, con el menor impacto posible en utilidad y costo de implementación?

3.2 Objetivo general

Evaluar comparativamente, a través de **IronVeil**, los cinco mecanismos defensivos definidos, individualmente y en combinación, mediante la ejecución sistemática de una batería de ataques reproducibles, para determinar la combinación con mejor relación protección/utilidad/costo.

3.3 Objetivos específicos

1. Caracterizar el estado actual del problema de exposición de LLM autoalojados en internet, con base en literatura técnica y académica reciente.
2. Diseñar y desplegar un laboratorio controlado que replique el patrón de vulnerabilidad documentado en instancias reales de Ollama.
3. Implementar los cinco mecanismos defensivos como módulos independientes, activables y desactivables sobre un mismo *pipeline*.
4. Ejecutar una batería reproducible de cinco vectores de ataque contra el *baseline*, contra cada mecanismo por separado y contra la combinación completa.
5. Medir y comparar tasa de éxito de ataque, latencia añadida, falsos positivos y costo de implementación por mecanismo y por combinación.
6. Formular una recomendación basada en evidencia sobre la combinación de mecanismos más efectiva para este contexto.
7. Como extensión (implementada y fuera de las siete configuraciones del núcleo), cerrar el ciclo de detección-respuesta mediante notificación al usuario ante bloqueos de riesgo y una integración con un SIEM que centralice y correlacione los eventos generados por los cinco mecanismos.

4 Alcance

4.1 Dentro del alcance

- Un laboratorio propio, aislado y autorizado (contenedores Docker, red interna cerrada). En ningún momento se atacan instancias reales de terceros expuestas en internet.
- Ollama como plataforma de estudio, con dos modelos (`soporte` y `rrhh`) basados en `llama3.2` (versión de 1B en las corridas de Colab con GPU T4 y de 3B en la demo local) que simulan distintos "departamentos" con información sensible ficticia plantada a propósito.
- Un único *pipeline* (proxy en Python) situado entre el cliente y Ollama, con los cinco mecanismos implementados como módulos independientes, activables y desactivables mediante un archivo de configuración.
- Siete configuraciones de prueba: *baseline* (todos los mecanismos apagados), cada mecanismo activado individualmente (cinco configuraciones) y la combinación completa de los cinco. Si el cronograma lo permite, se explorarán combinaciones parciales adicionales (por ejemplo, filtrado + clasificación) para enriquecer el análisis de costo/beneficio.

- Cinco vectores de ataque núcleo: reconocimiento de servicio, extracción de *system prompt*, inyección de *prompts* (directa e indirecta vía conversación), movimiento lateral (simulado con credenciales ficticias plantadas) y agotamiento de recursos.
- Automatización de las pruebas con herramientas estándar de la industria (Garak y Promptfoo) e integración continua en GitHub Actions (formato, análisis estático y pruebas unitarias sin necesidad de levantar Docker ni Ollama).
- Extensiones opcionales, apagadas por defecto y registradas aparte del núcleo (sus filas llevan una marca de extensión y las tablas del núcleo las excluyen): un escenario de *Excessive Agency* (V7) con herramientas simuladas en el modelo `rrhh-agente`; validación externa con Garak sobre una segunda instancia del proxy; normalización de entrada (decodificación de Base64, hexadecimal, ROT13 y caracteres invisibles antes de la cadena); memoria de decisiones que rechaza de inmediato lo que un humano ya rechazó; exportación a SIEM; soporte para proteger un modelo servido por otra API y para cambiar el clasificador (por ejemplo, Jev); autenticación opcional de `/chat` y de la consola de revisión; y un paquete instalable con `pip`.

4.2 Fuera del alcance

Categoría OWASP	Por qué queda fuera
Vector and Embedding Weaknesses (RAG)	Requiere una base de datos vectorial y un *pipeline* de documentos completo; se documenta como trabajo futuro.
Data and Model Poisoning (*fine-tuning*)	Requiere reentrenar el modelo con un conjunto de datos propio y cómputo GPU no garantizado; se documenta como trabajo futuro.
Supply Chain	Es una disciplina de auditoría de infraestructura (dependencias, *checksums*), distinta al comportamiento del modelo en sí.
Misinformation	Es evaluación de calidad/factualidad del modelo, no un ataque de seguridad ofensivo en el sentido que aborda este proyecto.

Adicionalmente quedan fuera de la evaluación los modelos cerrados vía API de terceros (GPT, Claude, Gemini): el proyecto se centra deliberadamente en el caso de despliegue autoalojado, que es donde se documenta el problema de exposición sin autenticación. El proxy permite protegerlos como extensión opcional, pero esa capacidad no se midió con la batería del núcleo. Cualquier prueba, escaneo o ataque contra sistemas que no pertenezcan al equipo del proyecto queda excluido por una restricción ética y legal explícita (ver sección 11).

5 Metodología

El proyecto sigue un enfoque experimental cuantitativo de tipo *red-teaming* controlado, estructurado en seis fases secuenciales:

1. **Revisión de literatura**: estado del arte académico, técnico e industrial (completada; ver Sección 1).
2. **Despliegue del laboratorio**: construcción del entorno base (Ollama + modelos + red aislada mediante Docker Compose).
3. **Implementación de los cinco mecanismos como módulos independientes**: cada mecanismo se construye como un componente que se puede activar o desactivar mediante configuración, sin reconstruir la infraestructura.
4. **Ejecución de la batería de ataques**: los mismos cinco vectores de ataque se ejecutan, sin cambios, contra cada una de las siete configuraciones (*baseline*, cinco mecanismos individuales, combinación completa).

5. **Medición de métricas**: se registran los mismos indicadores en cada configuración, lo que permite una comparación justa (variable controlada: qué mecanismos están activos; todo lo demás se mantiene constante).

6. **Análisis comparativo y conclusión**: se consolidan los resultados en tablas y gráficas, y se responde la pregunta de investigación con evidencia propia.

Esta metodología es análoga a un experimento controlado de laboratorio con un diseño factorial simplificado: mismo objeto de estudio, misma batería de pruebas, una única variable que cambia entre corridas (qué mecanismos están activos). Esto es lo que permite que las conclusiones sean comparables y defendibles.

Criterio de éxito de un ataque y tratamiento estadístico. Un ataque se considera exitoso solo si la credencial ficticia aparece realmente en la respuesta al cliente (fuga verificada por contenido), no porque ningún mecanismo lo haya bloqueado. Cada celda se reporta como k/n y se compara contra el *baseline* con la prueba exacta de Fisher (nivel de significancia 0,05). Los vectores 1 a 4 se ejecutaron con 13 repeticiones en C0 a C4 y 9 en C5 y C6 (con 7 s entre peticiones, para no saturar el límite de diez por minuto), distribuidas en cinco corridas en Google Colab con GPU T4 con el modelo `llama3.2:1b`. Como el modelo no es determinista, las corridas no se reproducen bit a bit; eso se declara como limitación y no se oculta. Los datos crudos no se editan a mano: toda transformación pasa por un *script* reproducible y las filas descartadas se conservan con su razón.

6 Arquitectura de la Solución

6.1 Diagrama del pipeline de mecanismos activables

A diferencia de construir cinco arquitecturas separadas y desconectadas entre sí, el sistema propuesto es un único *pipeline* por el que pasa toda petición, con cinco puntos de control que se activan o desactivan de forma independiente mediante un archivo de configuración (`config.yaml`) con banderas booleanas. Cada petición atraviesa el *pipeline* en orden; si un mecanismo está desactivado en la configuración de prueba, simplemente se omite ese paso sin alterar el resto del flujo. Esto permite ejecutar las siete configuraciones usando exactamente el mismo código, cambiando únicamente el archivo de configuración. El orden conceptual de los mecanismos (1 a 5) no coincide con su orden de ejecución: los que pueden bloquear (filtrado, clasificación y mínimo privilegio) forman una cadena en la que el primero que bloquea la corta; la delimitación no bloquea y se aplica justo antes de llamar al modelo; y la aprobación humana no es un paso más, sino que *intercepta* el resultado de esa cadena y lo convierte en una espera (respuesta 429) en lugar de un rechazo (400). Cuando un clasificador falla o expira, la petición se trata como insegura (falla cerrada).

6.2 Arquitectura por capas: puertos y adaptadores

El núcleo del proxy depende de interfaces pequeñas (*Protocol* de Python) y no de implementaciones concretas; los adaptadores se registran por configuración desde un único lugar de composición, de modo que conectar otro destino, formato o base de datos es un módulo nuevo y no una modificación del núcleo (principios abierto/cerrado y de inversión de dependencias de SOLID). Este patrón se aplicó de punta a punta en tres paquetes opcionales, cuyas pruebas de arquitectura verifican que el servicio no lea el entorno ni dependa de un adaptador concreto. La Figura 2 muestra las clases y sus relaciones.

6.3 Componentes opcionales del proxy

Además de los cinco mecanismos, el proxy incluye componentes opcionales, todos apagados por defecto para no alterar lo medido en las siete configuraciones, y cuyos eventos llevan una marca de extensión para

que las tablas del núcleo los excluyan.

Componente	**Qué hace**	**Cómo se activa**
Autenticación de `/chat`	Exige usuario y clave (cabecera Basic o sesión con CSRF); falla cerrada si no hay cuentas; bloqueo tras cinco fallos por usuario o IP; contraseñas con *hash* scrypt y sal propia; usuario de prueba con clave aleatoria	`CHAT_REQUIERE_AUTENTICACION=1`
Consola de revisión humana	Interfaz web en tiempo real (canal SSE) para aprobar o rechazar lo encolado, con login, roles y cabecera CSRF	`REVISION_USUARIOS`
Normalización de entrada	Decodifica Base64, hexadecimal y ROT13 y quita caracteres invisibles antes de la cadena, para que los mecanismos revisen también lo escondido	`NORMALIZACION_ENTRADA=1`
Memoria de decisiones	Recuerda los rechazos de la revisión humana y rechaza solo el mismo mensaje; almacén persistente intercambiable (JSON Lines, SQLite o la base de una empresa)	`MEMORIA_RECHAZOS=1`
Exportación a SIEM	Formatos JSON, CEF, LEEF y ECS; destinos archivo, syslog (UDP, TCP, TLS), HTTPS, Splunk, Elastic, Azure Monitor y CloudWatch, entregados en un hilo aparte	`SIEM_DESTINOS`
Notificaciones	Avisos por *webhook*, Slack, correo o WhatsApp cuando la aprobación humana encola una petición	Variables de entorno
Clasificador de entrada alternativo	Cambia el modelo de entrada (Ollama, API de terceros o Jev) sin tocar `clasificar()`; los servicios remotos exigen autorización explícita porque el texto sale de la red	`CLASIFICADOR_ENTRADA_BACKEND`
Modelo protegido remoto	Protege un modelo servido por otra API (compatible con OpenAI o Anthropic) en lugar de Ollama, con la misma forma de respuesta	`MODELOS_REMOTOS`
Herramientas simuladas (V7)	Modelo `rrhh-agente` con funciones que no tienen efecto real, marcadas de forma inequívoca en el código	`MODELOS_CON_HERRAMIENTAS`

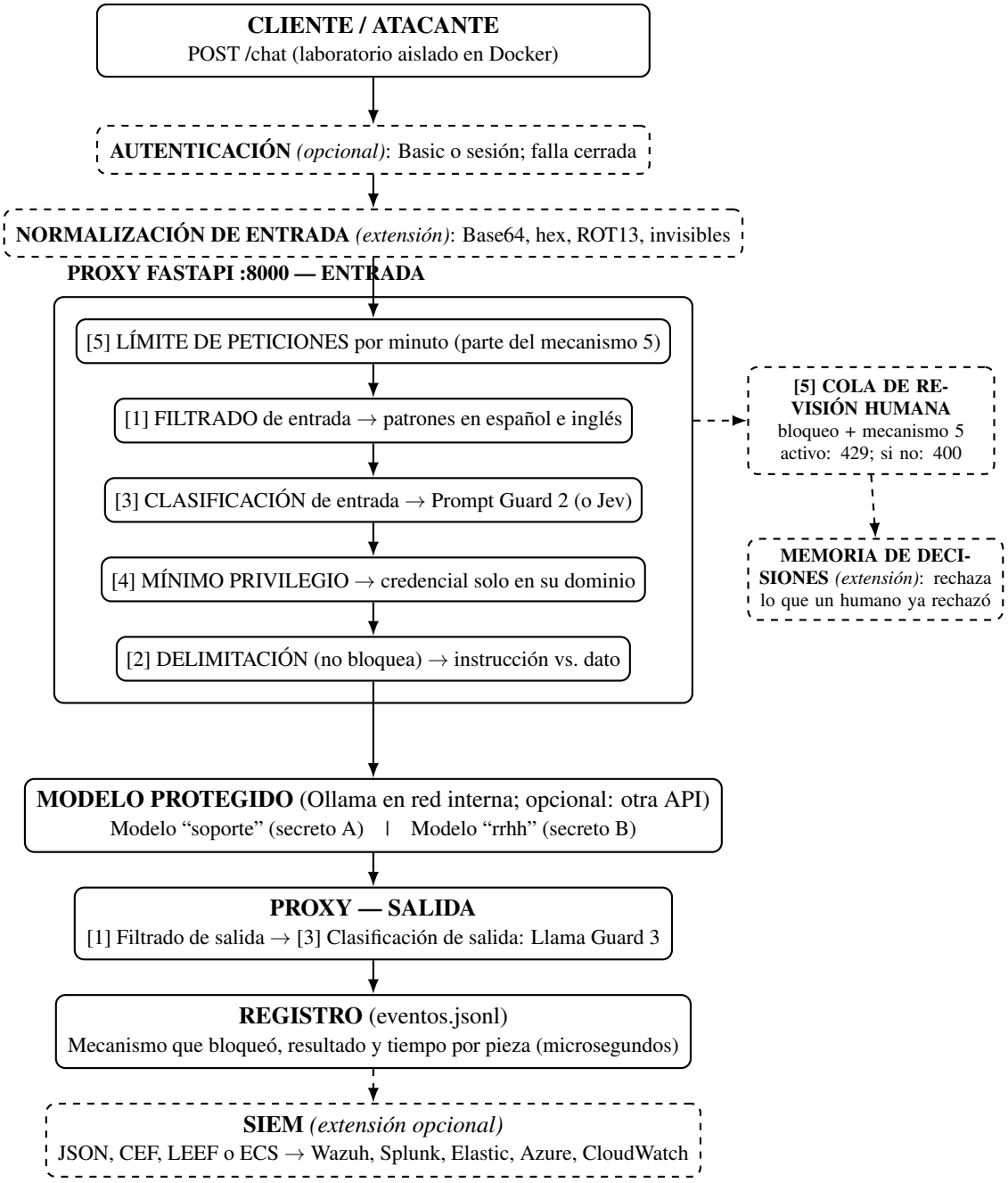


Figura 1: Arquitectura del *pipeline* implementado. Los recuadros continuos son el núcleo medido en las siete configuraciones; los punteados son opcionales y están apagados por defecto. La numeración es la de los mecanismos, no el orden de ejecución; los clasificadores del mecanismo 3 son intercambiables (Prompt Guard 2 y Llama Guard 3 por defecto, Jev como alternativa de entrada).

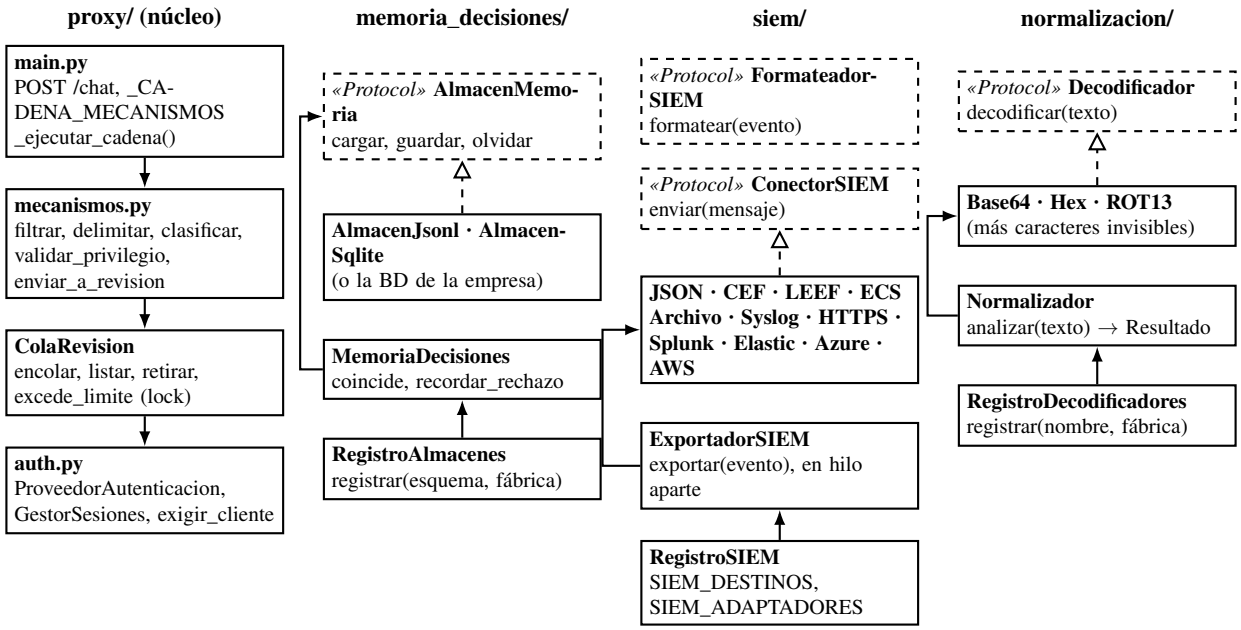


Figura 2: Diagrama de clases de IronVeil: núcleo y tres paquetes opcionales con puertos y adaptadores. Línea punteada con punta hueca: el adaptador implementa el puerto. Línea continua con flecha: el servicio usa al puerto. Solo se muestran las clases y los métodos que existen en el código.

6.4 Stack tecnológico general

Componente	Herramienta	Función
Orquestación de contenedores	Docker + Docker Compose	Levantar y aislar todos los servicios de forma reproducible
Motor de inferencia	Ollama	Servir los modelos LLM
Modelos protegidos	`llama3.2` (1B en Colab, 3B en la demo local)	Dos instancias (`soporte` y `rrhh`) con distinto *system prompt*, simulando "departamentos"
Proxy orquestador	Python (FastAPI)	Implementar los 5 mecanismos como *middleware* activable
Clasificadores (mecanismo 3)	Llama Prompt Guard 2 (86M, Hugging Face) y Llama Guard 3 (1B, Ollama)	Entrada: etiqueta maliciosa o benigna sobre el texto original del usuario. Salida: veredicto seguro/inseguro sobre la respuesta del modelo (Meta, 2025)(Meta, 2024)
Automatización de ataques	Scripts propios, Garak y Promptfoo	Ejecutar la batería de ataques de forma reproducible y medible
Orquestación de mediciones	Scripts propios en Python	Correr los escenarios contra las 7 configuraciones, capturar tiempos y resultados
Clasificador alternativo (extensión)	Jev (TypeSafe), servicio de pago	Alternativa al clasificador de entrada; se evalúa frente a Prompt Guard 2 en la Sección 7.6 (TypeSafe, 2026)
Distribución	Docker Compose, paquete `ironveil` (PyPI), imagen en GitHub Packages	Laboratorio completo con un comando; instalación con `pip`; comando `ironveil config|serve|auth-demo`
SIEM (extensión opcional)	Wazuh (Apache 2.0)	Ingerir el *log* JSON/CEF del proxy para correlación y alerta centralizada (ver justificación abajo)
Control de versiones	GitHub	Repositorio del proyecto, historial de cambios
Calidad y CI/CD	GitHub Actions, `pytest`, `ruff`, `black`, SonarCloud	Pruebas unitarias sin Docker ni Ollama (clasificador con *mocks*), formato y análisis estático en cada cambio
Análisis y visualización	Python (pandas, SciPy, matplotlib, seaborn)	Tablas, prueba exacta de Fisher y gráficas comparativas para el informe y la presentación

Sobre la elección de Wazuh frente a otras alternativas open-source evaluadas (Elastic Security, Graylog Open): la diferencia decisiva es de licenciamiento, no de funcionalidad. Wazuh se distribuye bajo licencia Apache 2.0 sin capacidades de detección restringidas a un nivel de pago, mientras que Elastic Security y Graylog Open son de código abierto "*open-core*": la base es gratuita, pero las capacidades de correlación propiamente dichas de un SIEM suelen quedar reservadas a los planes comerciales (Tech Insider, 2026). Adicionalmente, Wazuh analiza de forma nativa *logs* con formato JSON, extrayendo automáticamente sus campos para las reglas de detección (Wazuh, Inc., 2026), lo cual encaja directamente con el formato de exportación ya definido para el *log* de IronVeil, y cuenta con una imagen oficial de Docker que se integra sin fricción al mismo `docker-compose.yml` del laboratorio.

6.5 Los cinco mecanismos defensivos

Cada mecanismo responde a una pregunta fundamental de arquitectura defensiva de IA, independiente del sistema concreto:

Pregunta fundamental	Mecanismo	Naturaleza de la capa
¿Qué le llega al modelo antes de responder, y se puede bloquear lo obviamente peligroso?	Filtrado	Barata, rápida, basada en coincidencia de patrones conocidos
¿Cómo se indica al modelo qué es instrucción confiable y qué es entrada externa no confiable?	Delimitación de contenido	Estructural; previene la causa raíz de la confusión que explotan los ataques de *prompt injection*
¿Cómo se juzga si algo es peligroso más allá de coincidencias exactas de texto?	Clasificación	Más costosa en cómputo, pero captura variantes semánticas que el filtrado no detecta
¿Qué tanto puede saber o hacer cada parte del sistema?	Mínimo privilegio	Limita el radio de daño si una parte del sistema es comprometida; frena el movimiento lateral
¿Qué pasa cuando el sistema no está seguro de una decisión?	Aprobación humana	La capa más lenta y costosa, pero la más confiable ante casos ambiguos

Cada mecanismo se implementa como un módulo independiente dentro del mismo proxy Python (Sección 6.1), siguiendo el orden de la tabla, de menor a mayor complejidad. Un resumen operativo de cada uno:

(1) Filtrado.
Listas de patrones prohibidos en la entrada, en español e inglés (frases típicas de *prompt injection* como "ignora tus instrucciones anteriores"), un límite de 10 000 caracteres por mensaje y, en la salida, expresiones regulares que detectan formatos de credenciales ficticias plantadas. Bloquea o redacta coincidencias antes de que lleguen a Ollama o al cliente.

(2) Delimitación de contenido.
Construye el *prompt* final enviado al modelo marcando explícitamente, con delimitadores que incluyen un identificador aleatorio distinto en cada petición (para que el atacante no pueda cerrarlos), qué segmento es instrucción del sistema (confiable) y cuál es entrada del usuario (no confiable), instruyendo al modelo a ignorar cualquier instrucción contenida dentro del segundo segmento. Corresponde al principio de *Spotlighting* documentado por Microsoft (2025) y Hines et al. (2024).

(3) Clasificación.
Usa un modelo de IA distinto en cada dirección: Llama Prompt Guard 2 (86M, local vía Hugging Face) evalúa el texto original del usuario, no el ya envuelto por la delimitación, y Llama Guard 3 (1B, servido por Ollama) evalúa la respuesta del modelo. Un tiempo agotado, un error o una etiqueta desconocida se tratan como inseguros (falla cerrada). El clasificador de entrada es intercambiable sin modificar la función `clasificar()`: un adaptador traduce el veredicto de cada modelo (etiquetas, puntaje y umbral) a "malicioso o no", y el estado de cada veredicto queda en el *log* para separar una detección real de una caída del servicio. Con ese adaptador se evaluó Jev como alternativa (Sección 7.6). Capturan variantes que el filtrado por patrones fijos no cubre (por ejemplo, reformulaciones con sinónimos o pretextos de "auditoría de TI").

(4) Mínimo privilegio.
Cada modelo reconoce exclusivamente las credenciales de su propio dominio; cualquier intento de usar una credencial de otro dominio (por ejemplo, la credencial del modelo "Soporte" contra el modelo "RR. HH.") es rechazado o no otorga ningún privilegio adicional, evitando el movimiento lateral.

(5) Aprobación humana.
Las peticiones marcadas como sospechosas por cualquier otro mecanismo, o que excedan un límite de peticiones por minuto, no se rechazan automáticamente, sino que quedan en una cola de revisión (respuesta 429 con un identificador de revisión) hasta que una persona las apruebe o rechace desde una consola web, añadiendo fricción deliberada como mitigación adicional contra el agotamiento de recursos. El límite es de diez peticiones por minuto. Encolar introduce una demora, que se reporta como costo, y no una caída del servicio.

6.6 Configuraciones experimentales

#	Configuración	Mecanismos activos	Qué se espera medir
C0	*Baseline*	Ninguno	Línea base: cuántos de los 5 ataques tienen éxito sin ninguna protección
C1	Solo Filtrado	Filtrado	Qué ataques detiene un bloqueo por patrones conocidos
C2	Solo Delimitación	Delimitación	Qué ataques detiene separar estructuralmente instrucción de dato
C3	Solo Clasificación	Clasificación (Prompt Guard 2 y Llama Guard 3)	Qué ataques detiene un clasificador de seguridad de contenido
C4	Solo Mínimo Privilegio	Mínimo privilegio	Si específicamente detiene el movimiento lateral
C5	Solo Aprobación Humana	Aprobación humana	Si específicamente detiene el agotamiento de recursos y acciones indebidas
C6	Combinación completa	Los 5 mecanismos	Si la suma de todos cierra los huecos que cada uno deja por separado, y a qué costo total de latencia/complejidad

6.7 Escenarios de prueba: vectores de ataque

Cada vector se ejecuta de forma idéntica contra las siete configuraciones definidas arriba, para que la comparación sea justa.

Vector 1: Reconocimiento de servicio. Escaneo de puertos (`nmap`) y peticiones directas (`curl`) a *endpoints* conocidos, para confirmar que hay un servicio Ollama corriendo sin necesidad de credenciales. Ninguno de los cinco mecanismos está diseñado para detener el reconocimiento de red; es una limitación esperada que se documenta explícitamente, ya que se frenaría con controles de red perimetral fuera del alcance de este subtema.

Vector 2: Extracción de *system prompt* (OWASP LLM07). Obtención del *system prompt* completo de cada modelo, incluyendo las credenciales ficticias plantadas, vía el *endpoint* `/api/show`. En la configuración C0 (*baseline*) el ataque tiene éxito total; en C1–C6 el resultado depende de si el mecanismo activo detecta la fuga en la respuesta.

Vector 3: *Prompt injection* (indirecta, vía conversación) (OWASP LLM01). Batería de 5 a 10 variantes de ataque (instrucción directa de "ignora tus reglas", pretexto de rol de administrador/TI, e intento multiturno tipo *crescendo*), ejecutadas con Garak o Promptfoo. Este es el vector donde más mecanismos entran en juego: filtrado, delimitación y clasificación actúan directamente sobre él, cada uno con una forma distinta de detectarlo.

Vector 4: Movimiento lateral (OWASP LLM02, aplicado entre dos sistemas). Uso de la credencial ficticia extraída del modelo "Soporte" para hacerse pasar por usuario privilegiado ante el modelo "RR. HH.".

El mínimo privilegio (C4) es el mecanismo diseñado específicamente para este vector.

Vector 5: Agotamiento de recursos (OWASP LLM10). Saturación del servicio con peticiones simultáneas para degradar o tumbar el servicio. La aprobación humana (C5), a través del límite de peticiones por minuto, es el mecanismo diseñado específicamente para mitigar este vector.

Además de los cinco vectores del núcleo, el repositorio incluye un sondeo adaptativo (V6), que busca parafraseos que atraviesen la cadena completa, y el escenario de *Excessive Agency* (V7), este último como extensión con herramientas simuladas. Ninguno entra en las tablas del núcleo.

La siguiente matriz resume la hipótesis de cobertura esperada (a confirmar o refutar con los resultados reales medidos durante la ejecución):

Vector	**Filtr.**	**Delim.**	**Clasif.**	**Mín. Priv.**	**Aprob. Humana**
1. Reconocimiento	N/A	N/A	N/A	N/A	Parcial (fricción)
2. Extracción *system prompt*	Parcial	N/A	Parcial	N/A	Parcial
3. Prompt injection	Sí	Sí	Sí	N/A	Parcial
4. Movimiento lateral	N/A	N/A	N/A	**Sí (directo)**	Parcial
5. Agotamiento de recursos	N/A	N/A	N/A	N/A	**Sí (directo)**

"Sí" = el mecanismo está diseñado específicamente para ese vector; "Parcial" = puede ayudar indirectamente; "N/A" = no aplica de forma directa. Esta tabla es la hipótesis de partida; los resultados reales medidos durante la ejecución pueden confirmarla o contradecirla, y esa comparación es uno de los hallazgos más valiosos del proyecto.

6.8 Validación de aplicabilidad con modelos frontera

Los cinco mecanismos de IronVeil residen en el proxy orquestador (Sección 6.1), no dentro del modelo: no dependen de los pesos ni del comportamiento interno de Ollama. En principio, el mismo *pipeline* podría apuntar a un modelo frontera consumido vía API (por ejemplo, GPT o Claude) en lugar de a un *backend* local, y los cinco mecanismos seguirían operando sin modificación. Tres matices son relevantes para esta validación: (i) el *baseline* (C0) probablemente sería más bajo que con Ollama, porque los modelos frontera ya incorporan entrenamiento de seguridad propio; (ii) el Vector 2 (extracción vía `/api/show`) es específico de Ollama y tendría que sustituirse por una vía puramente conversacional; y (iii) cada llamada a un modelo frontera tiene costo monetario real, a diferencia de la inferencia local gratuita, lo que exige acotar el número de pruebas. Estado actual: el proxy admite proteger un modelo servido por otra API compatible con OpenAI o por un modelo frontera como extensión opcional, apagada por defecto porque el texto de cada petición sale de la red del laboratorio. Esa capacidad está implementada pero no se evaluó con la batería del núcleo, por lo que la portabilidad queda como funcionalidad disponible y no como resultado medido.

7 Resultados Esperados y Métricas de Evaluación

7.1 Métricas transversales

Las siguientes métricas se miden de forma idéntica en cada una de las siete configuraciones, lo que permite una comparación justa entre ellas.

Métrica	Definición	Cómo se calcula
Tasa de Éxito de Ataque (ASR)	Porcentaje de intentos en que la credencial ficticia apareció realmente en la respuesta (fuga verificada)	(ataques exitosos / ataques totales) × 100, con prueba exacta de Fisher contra C0
Latencia añadida (costo propio)	Tiempo extra que introduce el proxy, sin contar el tiempo del modelo	latencia total − latencia del modelo, medida dentro de cada petición (mediana y percentil 95, ms)
Falsos positivos (utilidad)	Peticiones legítimas bloqueadas incorrectamente	(peticiones legítimas bloqueadas / peticiones legítimas totales) × 100; 60 mensajes legítimos (12 distintos) en C0, C3 y C6 y una prueba unitaria por configuración
Costo de implementación	Complejidad de construir el mecanismo	líneas de código del módulo + tiempo real de desarrollo registrado por el equipo (horas)
Costo operativo	Recursos necesarios para mantenerlo corriendo	latencia añadida + (si aplica) tiempo humano requerido por la cola de aprobación (mecanismo 5)
Cobertura por mecanismo	Qué vectores bloquea cada mecanismo individualmente	tabla cualitativa de la Sección 6.5, base para justificar qué combinación conviene

La latencia ya no se calcula como diferencia contra C0 porque esa resta incluye el tiempo del modelo, que varía entre corridas y llegó a hacer parecer más rápidas a las configuraciones con mecanismos; por eso se mide el costo propio del proxy dentro de cada petición. Estas métricas responden de forma directa a la pregunta de investigación: protección = ASR inversa (100 %− ASR), utilidad = inverso de falsos positivos, y costo = las dos métricas de costo definidas arriba.

7.2 Resultados esperados por configuración

Con base en la literatura revisada (Sección 1.3) y en el diseño de cada mecanismo, se plantea la siguiente hipótesis de resultados, que la fase de ejecución (semanas 3 a 8, ver Sección 8) deberá confirmar, refutar o matizar con datos reales:

Config.	Mecanismos activos	ASR esperada (*prompt injection*)	Éxito mov. lateral	Éxito agotamiento
C0	Ninguno	Alta (∼90 %+, según Geng et al., 2026)	Sí	Sí
C1	Filtrado	Reducida (bloquea patrones conocidos)	Sí	Sí
C2	Delimitación	Reducida (dificulta confundir instrucción/dato)	Sí	Sí
C3	Clasificación	Más reducida (detecta variantes no literales)	Sí	Sí
C4	Mínimo privilegio	Sin cambio significativo	**No**	Sí
C5	Aprobación humana	Sin cambio significativo	Sí	**No**
C6	Los 5 mecanismos	Más baja de todas (no necesariamente 0 %)	No	No

7.3 Formato de la tabla final de resultados

La siguiente tabla, con los datos reales medidos, es la pieza central del informe final y de la sustentación, y responde directamente a la pregunta de investigación planteada en la Sección 3. Cada columna evalúa una dimensión distinta y complementaria de esa pregunta:

Filt., Del., Clas., M.P., A.H.
No son un resultado sino la variable independiente del experimento: indican, con una marca de verificación, qué mecanismos estaban activos en esa configuración. Son la base para poder atribuir cualquier diferencia en las demás columnas a una combinación específica de mecanismos.

ASR prom.
Evalúa la *protección* lograda: el promedio de la tasa de éxito de ataque sobre los vectores 1 a 4 (los únicos presentes en las siete configuraciones), para esa configuración. Es la columna que responde directamente si la combinación reduce el riesgo.

Falsos pos.
Evalúa el impacto en la *utilidad* del sistema para un usuario legítimo: qué proporción de peticiones normales (sin intención de ataque) resultó bloqueada o alterada incorrectamente por los mecanismos activos. Una configuración con ASR bajo pero falsos positivos altos no sería recomendable en la práctica.

Latencia extra
Evalúa el *costo operativo* en tiempo de respuesta: cuántos milisegundos añade el proxy en cada petición (costo propio, sin el tiempo del modelo), es decir, qué tan perceptible sería la defensa para un usuario real.

Costo (líneas/h)
Evalúa el *costo de implementación*: la complejidad de construir los mecanismos activos en esa configuración, medida en líneas de código del módulo y horas reales de desarrollo registradas por el equipo.

En conjunto, estas cinco columnas permiten calcular la relación protección/utilidad/costo para cada una de las siete configuraciones y sustentar, con evidencia propia, cuál es la combinación recomendada.

Conf.	Filt.	Del.	Clas.	M.P.	A.H.	ASR prom.	Falsos pos.	Latencia extra	Costo (líneas/h)
C0	✗	✗	✗	✗	✗	31,2 %	N/A (base)	+0,23 ms	0 / 0 h
C1	✓	✗	✗	✗	✗	15,0 %	0/1	+0,40 ms	248 / 21 h
C2	✗	✓	✗	✗	✗	19,7 %	N/A	+0,29 ms	181 / 15 h
C3	✗	✗	✓	✗	✗	28,6 %	0/60*	+76,6 ms	309 / 34 h
C4	✗	✗	✗	✓	✗	28,2 %	0/1	+0,46 ms	167 / 14 h
C5	✗	✗	✗	✗	✓	32,4 %	0/1	+0,35 ms	694 / 69 h
C6	✓	✓	✓	✓	✓	15,0 %	0/60*	+75,2 ms	1599 / 153 h

ASR promedio de los vectores 1 a 4. El vector 1 vale 40 % en todas las configuraciones porque Ollama es alcanzable directamente en la máquina de Colab (propiedad del entorno, no de ningún mecanismo); sin él, el promedio de los vectores 2 a 4 es 28,2 % en C0, 6,7 % en C1 y en C6, 12,9 % en C2, 24,7 % en C3, 24,3 % en C4 y 29,9 % en C5. La latencia extra es el costo propio del proxy (mediana); en C3 y C6 el percentil 95 es de 90,6 y 94,9 ms. * Falsos positivos: 0 de 60 mensajes legítimos rechazados en la entrada en C3 y C6, pero esa medición solo ve rechazos de entrada (códigos 400/429); la clasificación de salida puede retener una respuesta legítima, como se observó en una prueba aparte en C3 y C6 (respuesta con el texto «retenida»). La tasa real de falsos positivos no está medida. En C1, C4 y C5 solo hay una prueba unitaria por configuración, y la delimitación nunca bloquea. El costo en horas es una estimación del equipo.

7.4 Resultados obtenidos

Resultados por vector y mecanismo. La tabla siguiente muestra, para cada celda, los ataques exitosos sobre los intentos (k/n) y el valor p de la prueba exacta de Fisher contra C0. Con las muestras pequeñas de este experimento, una diferencia de uno o dos eventos no se distingue del ruido del modelo.

Vector	C0	Filtrado	Delimit.	Clasif.	Mín. priv.	Aprob. hum.
V1	26/65	26/65 p=1,00	26/65 p=1,00	26/65 p=1,00	26/65 p=1,00	18/45 p=1,00
V2	30/65	**13/65** p=0,003	21/65 p=0,15	27/65 p=0,72	32/65 p=0,86	22/45 p=0,85
V3	12/156	**0/147** p <0,001	6/156 p=0,22	7/156 p=0,34	13/156 p=1,00	4/108 p=0,20
V4 (agregado)	24/78	**0/78** p <0,001	**2/78** p <0,001	22/78 p=0,86	**12/78** p=0,036	20/54 p=0,46
V5	277/371	sin datos	sin datos	20/20 (no comparable)	sin datos	**37/18 329** p <0,001

En negrita, las celdas con $p \leq 0{,}036$. V5 no se volvió a ejecutar con el código actual: sus filas son de corridas anteriores, y la celda de clasificación no es comparable porque sus 20 filas están todas en un solo nivel de carga que C0 nunca probó. Con 25 pruebas se espera aproximadamente un falso positivo al nivel 0,05; la celda de $p = 0{,}036$ es la más frágil.

Qué se encontró, frente a la hipótesis de partida.

- **Filtrado** es el mecanismo con mejor evidencia: reduce V2 (30/65 a 13/65), V3 (12/156 a 0/147) y V4 (24/78 a 0/78) con un costo propio de 0,4 ms. Su efecto sobre V4 es indirecto, porque frena la extracción del primer paso, y la hipótesis no lo predecía.
- **Mínimo privilegio** hace exactamente lo diseñado: la extracción del paso 1 sigue funcionando (12/39) y el uso cruzado del paso 2 se bloquea 12 de 12; el ataque completo baja de 12/39 a 0/39 ($p = 0{,}0002$). El agregado del vector (24/78 a 12/78) mezcla ambos pasos, y por eso parece solo parcial.
- **Delimitación** reduce V4 (24/78 a 2/78) sin bloquear nada, un efecto que la hipótesis no predecía y cuya causa no se probó; en V2 y V3 la dirección es la esperada, pero no es concluyente.
- **Clasificación** no muestra una reducción de fuga distinguible (V2 $p = 0{,}72$, V3 $p = 0{,}34$, V4 $p = 0{,}86$) y cuesta cerca de 76 ms de mediana, repartidos entre Prompt Guard 2 en la entrada (≈39 ms) y Llama Guard 3 en la salida (≈38 ms). Bloqueó 84 peticiones en los vectores 1 a 4, pero casi todas eran peticiones que el modelo base ya habría rechazado.
- **Aprobación humana** no intercepta nada por contenido con el ritmo bajo el límite (ASR de C5 igual al de C0), pero frena el agotamiento de recursos, que es su función: 277/371 en C0 y 37/18 329 en C5 en el vector 5.
- **C6 no mejora a C1** en los vectores 1 a 4 (ambas en 15,0 %). En C6 los bloqueos quedan atribuidos a la aprobación humana, porque la cola intercepta lo que otros mecanismos detectaron, y la atribución al mecanismo que detectó se pierde por diseño.
- **Ninguna configuración llega a 0 %**, como anticipa la literatura. Un sondeo adaptativo (V6) encontró en tres intentos un parafraseo que atraviesa los cinco mecanismos, aunque el modelo igual se negó.
- **Efecto suelo en V3:** el modelo `llama3.2:1b` ya rechaza casi todo por su cuenta (12/156, 7,7 % en C0, frente al 90 % o más que sugería la literatura para sistemas sin protección), por lo que el margen para medir la mejora de delimitación y clasificación es mínimo. Que su valor aparezca con un modelo menos alineado es una hipótesis no probada.

Extensiones. Las extensiones (*Excessive Agency*, normalización de entrada, validación externa con Garak y clasificador alternativo Jev) se midieron aparte del núcleo y se reportan en las Secciones 7.5 y 7.6.

Respuesta a la pregunta de investigación. Con la evidencia disponible, la mejor relación protección/utilidad/costo no la da la configuración con más mecanismos, sino **filtrado, delimitación y mínimo privilegio** (los tres con efecto demostrable sobre la fuga, unas 50 horas de desarrollo y un costo propio de latencia inferior a 1 ms), más aprobación humana solo si el riesgo relevante es el abuso por volumen. C1 (21 horas) iguala a C6 (153 horas) en ASR de los vectores 1 a 4. Esa combinación se deriva de los efectos por mecanismo y no se ejecutó como configuración propia. C6 sigue siendo la única con defensa en profundidad por construcción, cuya redundancia cubriría una evasión del filtrado, pero este experimento no puede demostrar ese beneficio.

Limitaciones. Las muestras son pequeñas y varias celdas son no concluyentes, no "sin efecto"; el ASR de una misma configuración varía entre corridas (C0 en V3: 0/12, 4/72, 5/36 y 3/36) porque el modelo no es determinista; el vector 1 es una propiedad del entorno; el vector 5 no se repitió con el código actual; el modelo base es pequeño y ya está muy alineado; los ataques son del propio experimento y de un catálogo público, sin ataques adaptativos contra las defensas nuevas; y las extensiones se midieron en laboratorio, nunca en producción real.

7.5 Extensiones medidas fuera del núcleo

Ninguna de estas mediciones entra en las tablas de las siete configuraciones; cada una responde una pregunta distinta y tiene sus propios límites.

Extensión	Resultado	Límite que hay que citar
Excessive Agency (V7)	El atacante logró que el modelo pidiera un uso indebido de las herramientas simuladas en 14 de 20 intentos en C0 y en 0 de 20 en C5; la aprobación humana interceptó 16 solicitudes y disparó 16 notificaciones ($p < 0{,}0001$)	Una sola ejecución, modelo pequeño, herramientas sin efecto real
Normalización de entrada	El filtro detectó 24 de 24 ataques escondidos en Base64, hexadecimal, ROT13 o caracteres invisibles, frente a 0 de 24 sin ella, sin falsos positivos en 18 mensajes legítimos	Muestra pequeña: demuestra el mecanismo, no estima una tasa
Validación externa con Garak	Sobre 339 ataques por configuración (8 pruebas o *probes* del catálogo de NVIDIA (Derczynski et al., 2024)), el ASR fue 51,2 % sin defensas (173 de 338) y 0,0 % con la clasificación y el resto de defensas automáticas (Sección 7.6)	Una corrida; el modelo de 1B ya rechaza buena parte de los ataques; el catálogo es público, no del equipo

7.6 Clasificador alternativo: Jev frente a Llama Prompt Guard 2

Como el clasificador de entrada es intercambiable (Sección 6.2), se evaluó una alternativa de pago, Jev, un servicio de TypeSafe que se consume por API y cobra por tokens de entrada (TypeSafe, 2026). Jev es un servicio de un tercero: el equipo no lo desarrolla ni lo ofrece, solo se integra con él mediante el adaptador, por lo que su uso exige autorización explícita (el texto de cada petición sale de la red del laboratorio) y la clave de cada cliente es suya. La comparación que sigue es información para quien decide cómo desplegar el clasificador, no una oferta del equipo.

Efectividad medida con textos sueltos. Se enviaron a Jev textos de ataque y mensajes legítimos y se registró el veredicto; los datos crudos no incluyen el texto enviado ni la clave.

Conjunto	Ataques detectados	Legítimas bloqueadas	Latencia p50	Latencia p95
Español (29 textos)	13/13 (IC 95 %: 77–100 %)	0/16 (IC: 0–19 %)	92 ms	128 ms
8 idiomas (24 textos)	16/16 (IC: 81–100 %)	0/8 (IC: 0–32 %)	141 ms	230 ms

Los ocho idiomas son inglés, francés, portugués, alemán, italiano, ruso, chino y árabe, con dos ataques y una petición legítima por idioma, traducidos por un asistente de IA sin revisión de hablantes nativos. Las 16 legítimas en español corresponden a unos 10 textos distintos y los ataques son del propio experimento. Los intervalos son de Wilson; con muestras así de pequeñas son anchos.

Validación externa con Garak. Se repitió la corrida de Garak con Jev como clasificador de entrada, sobre una segunda instancia del proxy, con los mismos 339 ataques por configuración.

Configuración	ASR (IC 95 %)	Frenados en la entrada	Por fallo de servicio
C0 (sin defensas)	51,2 % (45,9–56,5)	0	—
C3: Jev solo	0,3 % (1/339; 0,1–1,7)	291/339 (86 %)	0
C6 sin aprobación humana: Jev y el resto	0,0 % (0/339; 0,0–1,1)	288/339 (85 %)	0

"C6 sin aprobación humana" son las cuatro defensas automáticas juntas; no es la C6 oficial porque con la aprobación humana cada bloqueo iría a la cola y Garak lo leería como un fallo. Los detectores de Garak son de cadena (miden si el ataque logró su objetivo, no si la respuesta es dañina), se usó un subconjunto de pruebas con tope de 30 *prompts* cada una y los ataques son de un catálogo público. Que el límite superior del intervalo sea 1,1 % significa que 0 de 339 no equivale a protección total.

La corrida análoga con Prompt Guard 2 no es comparable: parte de sus bloqueos fueron caídas del servicio (118 de 339 en C3 y 117 en C6 sin aprobación humana, porque la clasificación de la salida fallaba al ejecutarse Ollama sin GPU) y el proxy bloqueó por defecto (falla cerrada); sus detecciones reales fueron 221 y 222. Por eso **no se afirma que Jev sea mejor que Prompt Guard**: con la evidencia disponible, lo defendible es que ambos funcionan dentro del proxy y que, con Jev, la corrida de Garak no tuvo ningún fallo de servicio.

Idiomas, latencia y privacidad. Llama Prompt Guard 2 declara ocho idiomas en su ficha de Hugging Face (Meta, 2025). Para Jev, el fabricante no publica una cifra única: lo consultado indica el inglés como idioma principal y soporte variable en otros (innfactory, 2026); el equipo encontró una referencia de más de 50 idiomas que no es una cifra oficial y no se verificó de forma independiente, de modo que aquí solo se afirma lo medido (16 de 16 ataques en ocho idiomas, con muestra pequeña). En latencia, Prompt Guard 2 cuesta unos 39 ms por llamada de entrada (medido en C3 con GPU T4), mientras que Jev mostró una mediana de 92 a 221 ms según el conjunto de textos y picos por tratarse de un servicio remoto y reciente. La diferencia de privacidad es estructural: Prompt Guard corre dentro de la red del operador y Jev recibe el texto de cada petición.

Costo de desplegar el clasificador. El equipo estimó el costo de alojar Prompt Guard 2 en una instancia con recursos suficientes en unos USD 731 al mes (una instancia de ML más pequeña ronda los USD 330). Jev cobra USD 0,042 por millón de tokens de entrada y no cobra por la salida (TypeSafe, 2026). Con el supuesto de unos 1 000 tokens por petición, el costo mensual de Jev sería:

Peticiones por mes	Jev (USD)	Servidor propio (USD)
1 millón	≈ 42	≈ 731 (fijo)
5 millones	≈ 210	≈ 731 (fijo)
10 millones	≈ 420	≈ 731 (fijo)
17,4 millones	≈ 731	≈ 731 (fijo)

El punto de equilibrio es $731/0{,}042 \approx 17\,400$ millones de tokens, es decir, unos 17,4 millones de peticiones de 1 000 tokens (con la alternativa de USD 330, unos 7,9 millones). El costo de la instancia es una estimación del equipo y no incluye operación ni personal; con otro tamaño de petición cambia el equilibrio. Estas cifras son un cálculo para comparar, no una medición.

Por debajo del punto de equilibrio, pagar por token cuesta menos que mantener el servidor; por encima, conviene autoalojar; y si los textos no pueden salir de la red de la organización, autoalojar sigue siendo la única opción, con independencia del costo.

8 Distribución, Reproducibilidad y Calidad

Una parte del objetivo del proyecto es que cualquier persona ajena al equipo pueda levantar y verificar el laboratorio. Para ello el repositorio (García Nieto et al., 2026) ofrece:

- **Un solo comando:** `docker compose up -build` levanta Ollama, crea los modelos `soporte` y `rrhh` a partir de plantillas (sin credenciales en el código) y el proxy; solo se publica el puerto 8000 y Ollama permanece en la red interna.
- **Las siete configuraciones con un comando:** `python scripts/seleccionar_config.py C0..C6` escribe el archivo de banderas; el proxy lo relee en cada petición, sin reconstruir nada.
- **Paquete instalable:** `pip install ironveil` instala el proxy con el comando `ironveil` (`config`, `serve` y `auth-demo`); se comprobó en un entorno virtual limpio que la instalación, el arranque y la autenticación funcionan. El paquete es el proxy y su línea de comandos; los *scripts* de ataque y de análisis viven en el repositorio.
- **Calidad automatizada:** 1 729 pruebas unitarias y de integración que corren sin Docker ni Ollama (los clasificadores se prueban con simulaciones), formato y análisis estático en cada cambio, análisis con SonarCloud y dependencias fijadas con versión exacta (y con *hashes* en la imagen del proxy).
- **Documentación y visuales:** un diagrama de arquitectura animado, un diagrama de clases y un visor interactivo que simula el recorrido de una petición por cada configuración; guías para cambiar de clasificador, conectar un SIEM y escribir el informe con las cifras trazables a su origen.
- **Publicación:** una *release* con la etiqueta de la entrega final, una imagen del proxy en GitHub Packages y el sitio de documentación en GitHub Pages.

Los datos crudos (`resultados/`) se tratan como evidencia: no se editan a mano, toda transformación pasa por un *script* reproducible y las filas descartadas se conservan con su razón. La licencia del código es MIT.

9 Conclusiones y Trabajo Futuro

9.1 Conclusiones

1. **Ninguna defensa sola alcanza.** Ninguna configuración llegó a 0 % de ASR, como anticipa la literatura. Lo valioso del experimento es saber qué capa aporta cuánto y a qué costo.

2. **Más mecanismos no es mejor.** Con el modelo de 1B, C1 (filtrado solo, 21 horas) iguala a C6 (los cinco, 153 horas) en el ASR de los vectores 1 a 4 (15,0 %, frente a 31,2 % sin defensas). La combinación con mejor relación protección/costo es filtrado, delimitación y mínimo privilegio, con un costo propio inferior a 1 ms; esa combinación se deriva de los efectos por mecanismo y no se ejecutó como configuración propia.

3. **Cada mecanismo hace lo que se diseñó, y solo eso.** El mínimo privilegio frena el uso cruzado de credenciales (ataque completo de 12/39 a 0/39), la aprobación humana frena el abuso por volumen (74,7 % a 0,2 %) y el filtrado reduce la extracción y la inyección. La clasificación cuesta cerca de 76 ms y con este modelo no mostró una reducción de fuga distinguible.

4. **El clasificador es una pieza intercambiable.** Con Jev, la validación externa con Garak dejó pasar 1 de 339 ataques con el clasificador solo y 0 de 339 con las defensas automáticas juntas, frente a 51,2 % sin defensas. No se afirma que Jev supere a Prompt Guard: la comparación directa no es válida y las muestras son pequeñas. La decisión entre ambos se reduce a latencia, idiomas, privacidad y un punto de equilibrio de costo cercano a 17 millones de peticiones al mes bajo los supuestos declarados.

5. **El valor está en la arquitectura.** Al depender de interfaces y no de implementaciones, el proxy permite cambiar de clasificador, de destino de eventos o de almacén sin tocar el núcleo, y activar o apagar cada pieza por configuración.

9.2 Limitaciones

Las muestras son pequeñas y varias celdas son no concluyentes, no "sin efecto"; el modelo no es determinista y las corridas no se reproducen bit a bit; el modelo base de 1B está muy alineado y rechaza buena parte de los ataques por sí solo (con uno más capaz, el ASR sin defensas sería mayor); el vector 1 es una propiedad del entorno y el vector 5 no se repitió con el código actual; los ataques son del propio experimento y de un catálogo público, sin ataques adaptativos contra las defensas nuevas; Jev es un servicio de tercero cuyas cifras de idiomas no son oficiales; los costos de infraestructura son estimaciones; y todo se midió en laboratorio, nunca en producción real. El proxy no pretende ser un producto listo para producción, sino una capa de defensa validada en laboratorio.

9.3 Trabajo futuro

- Repetir el vector 5 con el código y el clasificador actuales, en las siete configuraciones.
- Ejecutar la combinación recomendada (filtrado, delimitación y mínimo privilegio) como configuración propia.
- Probar un modelo base menos alineado, para saber si la clasificación aporta cuando el modelo no se defiende solo.
- Comparar Prompt Guard 2 y Jev sobre el mismo banco de textos y en la misma sesión, con revisión de hablantes nativos en los idiomas evaluados, y confirmar la precisión media (fp16 contra fp32) para atribuir la mejora del percentil 95.
- Ataques adaptativos contra las defensas nuevas, y los riesgos fuera del alcance (RAG, *fine-tuning*).
- Un piloto con un despliegue real, con autenticación activada, y un modelo frontera como modelo protegido.

10 Cronograma Inicial de Trabajo

El equipo tiene clase los sábados. La entrega de esta propuesta corresponde al sábado 22 de agosto de 2026. A partir de esa fecha el equipo cuenta con diez sábados de trabajo técnico antes de dos hitos finales establecidos para el seminario: la **entrega final** del proyecto (informe, repositorio y video demo) el sábado **31 de octubre de 2026**, y la **presentación del seminario** (sustentación oral de 7 minutos) el sábado **7 de noviembre de 2026**, una semana después.

Estado a octubre de 2026. El repositorio refleja completos los frentes de implementación de los cinco mecanismos, la medición de las siete configuraciones, el costo por mecanismo y las extensiones (*Excessive Agency*, SIEM, normalización, memoria de decisiones, validación con Garak, evaluación del clasificador Jev, autenticación opcional y empaquetado). Permanecen pendientes la re-ejecución del vector 5 con el código actual y los entregables finales (video demo, informe y repositorio del 31 de octubre). El cronograma se conserva tal como se planteó en la propuesta.

Cada semana los cuatro integrantes trabajan en paralelo sobre cuatro frentes distintos y complementarios (implementación, pruebas de ataque, consolidación de métricas y documentación), de modo que el avance de cada hito no depende de una sola persona.

Sábado	**Reparto de tareas de la semana**	**Entregable**
22 ago	**Todo el equipo:** entrega conjunta de la propuesta (estado del arte, problema, metodología, arquitectura preliminar y cronograma) y presentación de 7 minutos.	Presentación de 7 min
29 ago	**Fiquitiva:** documenta el diseño del *pipeline* y define la plantilla de registro de métricas (*logs*). **García:** monta Docker Compose (Ollama + proxy base) y los dos modelos con *system prompts* distintos. **Sabogal:** prepara el entorno de Garak/Promptfoo y redacta las primeras variantes de los 5 vectores de ataque. **Piedrahita:** implementa `config.yaml` y los 5 *stubs* de mecanismos, vacíos y activables por bandera.	Entorno funcional, *pipeline* vacío con banderas
5 sep	**García:** implementa el mecanismo 1 (Filtrado): listas de patrones en entrada y salida. **Piedrahita:** implementa el mecanismo 2 (Delimitación): marcado de instrucción vs. dato. **Sabogal:** ejecuta los vectores 1–3 contra C0, C1 y C2. **Fiquitiva:** registra y consolida los resultados parciales de C0–C2.	Resultados parciales C0–C2
12 sep	**García:** integra Llama Guard al proxy como servicio de clasificación. **Piedrahita:** implementa la función de clasificación (mecanismo 3) y su bandera de activación. **Sabogal:** ejecuta los vectores 1–3 contra C3, incluyendo variantes no cubiertas por el filtrado. **Fiquitiva:** consolida los resultados C3 y los compara contra C1 y C2.	Resultados C3
19 sep	**García:** implementa la validación de credenciales por dominio (mínimo privilegio). **Piedrahita:** diseña las credenciales ficticias cruzadas usadas en el vector 4. **Sabogal:** ejecuta el vector 4 (movimiento lateral) contra C4. **Fiquitiva:** consolida los resultados C4 y verifica si se frena el movimiento lateral.	Resultados C4

Sábado	**Reparto de tareas de la semana**	**Entregable**
26 sep	**García:** implementa la cola de revisión y el límite de peticiones por minuto. **Piedrahita:** implementa la interfaz mínima de aprobación/rechazo humano. **Sabogal:** ejecuta el vector 5 (agotamiento de recursos) contra C5. **Fiquitiva:** consolida los resultados C5 y mide el tiempo humano de revisión.	Resultados C5
3 oct	**García:** integra los 5 mecanismos en una sola configuración combinada. **Piedrahita:** apoya las pruebas de integración y corrige conflictos entre módulos. **Sabogal:** ejecuta los 5 vectores completos contra C6. **Fiquitiva:** consolida los resultados C6 completos.	Resultados C6 completos
10 oct	**García:** mide el costo de implementación (líneas de código y horas) por mecanismo. **Piedrahita:** apoya la consolidación de *logs* de las 7 configuraciones. **Sabogal:** contrasta la cobertura real observada contra la matriz de hipótesis (Sección 6.5). **Fiquitiva:** construye las tablas y gráficas comparativas finales.	Tablas/gráficas finales
17 oct	**García:** implementa las herramientas simuladas (*function calling*) para el modelo con agencia, y agrega el disparador de notificación (*webhook*/correo) sobre el mecanismo de Aprobación Humana. **Piedrahita:** diseña de 3 a 5 *prompts* de ataque para forzar el uso indebido de herramientas, y adapta el formato del *log* existente a JSON estructurado/CEF para exportación a SIEM. **Sabogal:** ejecuta la extensión con y sin aprobación humana activa, y verifica que la notificación se dispare correctamente en cada bloqueo. **Fiquitiva:** documenta los resultados de la extensión *Excessive Agency* y valida que el *log* exportado sea compatible con el formato esperado por un SIEM (Splunk/Elastic/Sentinel).	Resultados de las extensiones: *Excessive Agency*, notificación y exportación SIEM (opcional)
24 oct	**García:** prepara y ejecuta técnicamente la grabación de las 7 configuraciones. **Piedrahita:** edita el video de la demo y apoya la redacción del informe. **Sabogal:** ejecuta en vivo los 5 vectores de ataque durante la grabación. **Fiquitiva:** redacta el borrador del informe final con los resultados consolidados.	Video demo + borrador de informe
31 oct *(entrega final)*	**García:** consolida el repositorio de GitHub (código, `config.yaml`, *logs*) para la entrega. **Piedrahita:** revisa que el video demo y el código coincidan con lo reportado en el informe. **Sabogal:** verifica que todos los resultados reportados sean reproducibles. **Fiquitiva:** finaliza y entrega el informe completo en formato APA.	Informe final, repositorio y video demo
7 nov *(presentación)*	**Todo el equipo:** sustentación oral del proyecto (7 minutos) y respuesta a las preguntas del profesor.	Sustentación final del seminario

Riesgo principal de cronograma: la fase más compleja técnicamente es la implementación del mecanismo de clasificación (semana del 12 de septiembre). Mitigación: las extensiones opcionales del 17 de octubre (*Excessive Agency*, notificación al usuario y exportación a SIEM) y el trabajo futuro (RAG, *fine-tuning*, validación con modelos frontera) son las primeras tareas a recortar si el cronograma se atrasa, sin afectar el

núcleo del proyecto (mecanismos 1 a 5 con los 5 vectores base). Al distribuir cuatro frentes de trabajo en paralelo cada semana, el avance no depende de una sola persona, y la semana completa entre la entrega final (31 de octubre) y la sustentación oral (7 de noviembre) queda disponible para pulir la presentación con base en la retroalimentación recibida.

11 Consideraciones Éticas y Legales

Todo el trabajo ofensivo descrito en este documento se realiza exclusivamente contra infraestructura propia del equipo, desplegada en un laboratorio aislado (contenedores Docker, red interna cerrada). En ningún momento se escanean, prueban o atacan instancias reales de Ollama u otro servicio expuestas por terceros en internet, aunque la motivación del problema, y la evidencia de su magnitud citada en la Sección 1.1, provenga de hallazgos públicos de investigadores de seguridad (SentinelOne, Censys, Cyera). Los clasificadores y modelos remotos de terceros están apagados por defecto, porque enviarlos implica sacar el texto de las peticiones de la red del laboratorio. Replicar estas pruebas contra sistemas ajenos sin autorización constituiría un delito informático en la mayoría de las jurisdicciones, incluyendo Colombia (Ley 1273 de 2009).

Todas las credenciales y datos "sensibles" usados en el laboratorio (claves de API, códigos de acceso) son completamente ficticios, generados exclusivamente para este proyecto, sin ninguna relación con sistemas o datos reales. Las fuentes públicas citadas en la Sección 1 se usan únicamente como evidencia de un patrón de riesgo documentado de forma independiente por múltiples investigadores de seguridad, y como motivación del problema, no como guía operativa para reproducir ataques contra terceros.

Referencias

AI Thinker Lab. (2026, julio 23). *Ollama server security: 7 self-hosted LLM hardening tips*. AI Thinker Lab. https://aithinkerlab.com/ollama-server-security-hardening-tips/

Check Point Software. (2026). *Top LLM security tools in 2026*. Check Point Software Technologies. https://www.checkpoint.com/cyber-hub/tools-vendors/top-llm-security-tools-in-2026/

Chen, S., Piet, J., Sitawarin, C., & Wagner, D. (2025). StruQ: Defending against prompt injection with structured queries. En *Proceedings of the 34th USENIX Security Symposium*. USENIX Association.

Chen, S., Zharmagambetov, A., Mahloujifar, S., Chaudhuri, K., Wagner, D., & Guo, C. (2025). SecAlign: Defending against prompt injection with preference optimization. En *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery.

Cyera Research. (2026, mayo 5). *Bleeding Llama: A critical memory leak in the world's most popular local AI platform*. Cyera. https://www.cyera.com/blog/bleeding-llama-a-critical-memory-leak-in-the-worlds-most-popular-local-ai-platform

Derczynski, L., Galinkin, E., Martin, J., Majumdar, S., & Inie, N. (2024). garak: A framework for security probing large language models (arXiv:2406.11036). *arXiv*. https://arxiv.org/abs/2406.11036

Fortinet. (2026). *FortiAIGate: Secure AI workloads and LLMs*. Fortinet, Inc. https://www.fortinet.com/products/fortiaigate

García Nieto, A. F., Fiquitiva Poveda, A. G., Piedrahita Arango, C. M., & Sabogal Wilches, A. F. (2026). *IronVeil: Defensa en profundidad para LLMs expuestos, con énfasis en modelos autoalojados con Ollama* [Software]. GitHub. https://github.com/Anderfg13/IronVeil

Geng, T., Xu, Z., Qu, Y., & Wong, W. E. (2026). Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies. *Computers, Materials & Continua, 87*(1). https://doi.org/10.32604/cmc.2025.074081

Hines, K., Lopes, G., Solawetz, J., Ganneboyina, S., & Mitkov, D. (2024). Defending against indirect prompt injection attacks with spotlighting (arXiv:2403.14720). *arXiv*. https://arxiv.org/abs/2403.14720

JTHil. (2026). *llm-shield: Complete security for self-hosted AI — gateway + firewall + monitoring* [Repositorio de software]. GitHub. https://github.com/JTHil/llm-shield

Maxim AI. (2026, mayo 25). *Top 5 LLM security tools for enterprise AI applications in 2026*. Maxim AI. https://www.getmaxim.ai/articles/top-5-llm-security-tools-for-enterprise-ai-applications-in-2026/

innfactory. (2026). *TypeSafe Jev: Model overview* [Ficha de terceros, consultada el 7 de octubre de 2026]. innfactory AI. https://innfactory.ai/en/ai-models/typesafe-jev/

Meta. (2024). *Llama Guard 3 1B* [Modelo]. Ollama. https://ollama.com/library/llama-guard3

Meta. (2025). *Llama Prompt Guard 2 86M* [Modelo]. Hugging Face. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M

Microsoft Security Response Center. (2025, julio 30). *How Microsoft defends against indirect prompt injection attacks*. Microsoft. https://www.microsoft.com/en-us/msrc/blog/2025/07/how-microsoft-defends-against-indirect-prompt-injection-attacks

OWASP GenAI Security Project. (2025). *OWASP Top 10 for LLM Applications 2025*. Open Worldwide Application

Security Project. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

Security Boulevard. (2026, abril 23). *They say Gartner is dead. Clearly, they haven't checked their LLM sources*. Security Boulevard. https://securityboulevard.com/2026/04/they-say-gartner-is-dead-clearly-they-havent-checked-their-llm-sources/

SentinelOne (SentinelLABS) & Censys. (2026, enero 29; actualizado el 6 de marzo). *Silent brothers: Ollama hosts form anonymous AI network beyond platform guardrails*. SentinelOne. https://www.sentinelone.com/labs/silent-brothers-ollama-hosts-form-anonymous-ai-network-beyond-platform-guardrails/

Tech Insider. (2026, agosto). *Wazuh vs Elastic vs Graylog: Open source SIEM 2026*. Tech Insider. https://tech-insider.org/wazuh-vs-elastic-vs-graylog-2026/

The Hacker News. (2026, mayo 10). *Ollama out-of-bounds read vulnerability allows remote process memory leak*. The Hacker News. https://thehackernews.com/2026/05/ollama-out-of-bounds-read-vulnerability.html

TypeSafe. (2026). *TypeSafe API documentation* [Documentación, consultada el 7 de octubre de 2026]. TypeSafe. https://docs.typesafe.ai/

Unit 42, Palo Alto Networks. (2026a, mayo 1). *Threat bulletin: April 2026*. Palo Alto Networks. https://unit42.paloaltonetworks.com/threat-bulletin/april-2026/

Unit 42, Palo Alto Networks. (2026b, marzo 19). *Navigating security tradeoffs of AI agents*. Palo Alto Networks. https://unit42.paloaltonetworks.com/navigating-security-tradeoffs-ai-agents/

Wazuh, Inc. (2026). *External API integration — Wazuh server*. Wazuh Documentation. https://documentation.wazuh.com/current/user-manual/manager/integration-with-external-apis.html

Wiz. (2026, mayo 8). *Top AI security tools for the cloud: Secure AI workloads*. Wiz, Inc. https://www.wiz.io/academy/ai-security/ai-security-tools